

Regular Expressions (Regex)

Patterns

- * : 0 or more
- + : 1 or more
- ? : 0 or 1
- . : any character
- ^: start of string
- \$: end of string
- {n} : exactly n times
- {n,} : n or more times
- {n,m} : between n and m times
- {,m} : up to m times
- [abc] : a or b or c
- [a-z] : any character from a to z
- [^abc] : any character except a or b or c
- \d : digit [0-9]
- \D : non-digit [^0-9]
- \w : word character [a-zA-Z0-9_]
- \W : non-word character [^a-zA-Z0-9_]
- .* : any character (0 or more times)

- Precedence order:**
 - Parentheses ()
 - Counters (*, +, ?, {}, etc)
 - Sequences/anchors (ab, ^, \$)
 - Disjunction (—)

Tokenization

- Top-down tokenization** (rule-based). For languages that use space characters between words.
- Bottom-up tokenization** (data-driven approach, e.g. BPE). Subword tokenization. Better at handling out-of-vocab words and rare words.
- Can't blindly remove punctuation symbols (e.g Ph.D, \$45.55, 01/04/2002, URLs)

Penn Treebank Tokenization Standard

- Seperate out clitics (doesn't → does n't)
- Keep hyphenated words together
- Separate out punctuation symbols

Byte-Pair Encoding

- Token learner.** Input: Raw training corpus. Output: Vocab (set of tokens)
 - Initialise vocab to set of all unique single characters: {A, B, C, ..., a, b, c}
 - Add special end-of-word symbol ' ' before space in training corpus
 - Note: Make all words character separated. Have a count of the words.
 - Choose 2 symbols that are most frequently adjacent in training corpus (say 'X', 'Y')
 - Add a new merged symbol 'XY' to the vocab
 - Replace every adjacent 'X', 'Y' in the corpus with 'XY'
 - Repeat until *k*-merges have been done
- Token segmenter.** Input: Raw test sentence and Vocab. Output: Tokenized sentence.
 - Make all words character separated.
 - Greedily merge characters in the order we learned them.

Normalization

- Lemmatization:** Represent all words as their lemma, their shared root (dictionary headword form). (e.g. am, is, are → be; car, cars, car's, cars' → car)
- Morphemes:**
 - Stems: The core meaning-bearing units
 - Affixes: parts that adhere to stems, often with grammatical function
- Porter Stemming Algorithm:** A series of rewrite rules run in a cascade. Reduces words to stems by chopping off affixes crudely. E.g (ATIONAL → ATE, ING → *ε* if stem contains vowel, SSES → SS)

Min-Edit Distance

- Source: Delete operation. Bottom up. Cost = 1
- Target: Insert operation. Left to right. Cost = 1
- Diagonal: Substitution operation. Diagnoal. Cost = 0 if same characters, 2 otherwise.

N-Gram Language Models

- Probability of a language sequence. Word Prediction.
- Use the previous *n* − 1 words to predict the next word.
$$P(w_n|w_{1:n-1})$$
$$P(w_{1:n}) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_{1:2}) \times \cdots \times P(w_n|w_{1:n-1}) = \prod_{k=1}^n P(w_k|w_{1:k-1})$$
- Estimating n-gram probability:
 - Maximum Likelihood Estimate (MLE)**
$$P(w_n|w_{1:n-1} = \frac{C(w_{w1:n})}{C(w_{w1:n-1})}$$
 (n-gram)
 - $P(w_2|w_1 = \frac{C(w_1w_2)}{C(w_1)}$ (bigram)
 - Data sparsity when estimating probability of longer n-grams
- N-gram approximation:
 - Markov assumption:
$$P(w_k|w_{1:k-1}) \approx P(w_k|w_{k-(N-1):k-1})$$
 - w_k* only depends on its previous *N* − 1 words (*N* ≤ *k*)
 - Bigram Approximation:
$$P(w_{1:n}) \approx \prod_{k=1}^n P(w_k|w_{k-1})$$
 - Use logprob instead as multiplying many probabilities results in numerical underflow

Evaluating N-Gram Models

- Extrinsic vs Intrinsic Evaluation

Perplexity

- $PP = m(w_1, \dots, w_n)^{-\frac{1}{n}} = \frac{1}{\sqrt[n]{m(w_1, \dots, w_n)}}$
- The **lower** the perplexity the better
- If perplexity = k, the model is uncertain as if it had to choose uniformly among k equally likely words at each step. Weighted average branching factor of a language.
- Bigram LM:
$$PP = \frac{1}{\sqrt[n]{m(w_1, \dots, w_n)}} = \frac{1}{\sqrt[n]{\prod_{k=1}^n P(w_k|w_{k-1})}}$$

Smoothing

- Lowering non-zero n-gram counts in order to assign some probability mass to zero n-grams
- Smoothed bigram count:
$$P(w|w_0) = \frac{C(w_0w)+1}{C(w_0)+V} = \frac{C^*(w_0w)}{C^*(w_0)}$$
- Discount $d = \frac{C^*(w_0w)}{C(w_0)}$

- Add-k smoothing for bigram: $P(w|w_0) = \frac{C(w_0w)+k}{C(w_0)+kV}$

Interpolation

- Combining benefits of higher order n-grams with lower-order n-grams
- Bigrams: $\hat{P}(w_0|w_{-1}) = \lambda_1P(w_0|w_{-1}) + \lambda_2P(w_0)$
- Trigrams: $\hat{P}(w_0|w_{-2}w_{-1}) = \lambda_1P(w_0|w_{-2}w_{-1}) + \lambda_2P(w_0|w_{-1}) + \lambda_3P(w_0)$
- Note: $\sum_i \lambda_i = 1$. Select λ_i that maximises the probability of the development set.

Entropy

- $H(X) = -\sum_{x \in X} p(x) \log_2 p(x)$
- Number of bits to encode information in the optimal coding scheme
- Entropy of a sequence in language *L*:
$$H(w_1, w_2, \dots, w_n) = -\sum_{w_{1:n} \in L} p(w_{1:n}) \log_2 p(w_{1:n})$$
- Entropy rate (per-word entropy):
$$\frac{1}{n} H(w_{1:n}) = -\frac{1}{n} \sum_{w_{1:n} \in L} p(w_{1:n}) \log_2 p(w_{1:n})$$
- $H(L) = \lim_{n \rightarrow \infty} -\frac{1}{n} \sum_{w_{1:n} \in L} p(w_{1:n}) \log_2 p(w_{1:n})$
- By Shannon-McMillan-Breiman theorem:
$$H(L) = \lim_{n \rightarrow \infty} -\frac{1}{n} \log_2 p(w_1, \dots, w_n)$$
- Cross Entropy:** Used for comparing two language models, *m* and *p*
$$H(p, m) = \lim_{n \rightarrow \infty} -\frac{1}{n} p(w_{1:n}) \log_2 m(w_{1:n})$$
- By Shannon-McMillan-Breiman theorem:
$$H(p, m) = \lim_{n \rightarrow \infty} -\frac{1}{n} \log_2 m(w_{1:n})$$
- Property: $H(p) \leq H(p, m)$. Difference between them is a measure of how accurate model *m* is.
- More accurate the model, the **lower its cross-entropy**
- Perplexity (PP)** = $2^H = 2^{-\frac{1}{n} \log_2 m(w_{1:n})}$

Paired Bootstrap Test

- Create a test set *x_i* of the same size as *x* by sampling from *x* with replacements
- Calculate $\delta(x_i) = M(A, x_i) - M(B, x_i)$
- $s = s + 1$ if $\delta(x_i) \leq 0$
- p-value = $\frac{s}{b}$. *b* is num of samples
- If p-value < 0.01, then A is better than B at 0.01 level of significance

Linear Models

- $y = f(x) = x \cdot W + b$
- $x \in \mathbb{R}^{d_{in}}; W \in \mathbb{R}^{d_{in} \times d_{out}}; y, b \in \mathbb{R}^{d_{out}}$
- In the matrix representation, add a 1 to the *x* input and *b* to the last row of *W*
- Binary classification: Perceptron.
$$class(x) = \hat{y} = 1(\text{if } f(x) \geq 0); 0(\text{if } f(x) < 0)$$
- Sigmoid function:** maps numbers to 0-1. $\sigma(x) = \frac{1}{1+e^{-x}}$
- Logistic regression:**
 - $P(\hat{y} = 1|x) = \sigma(x \cdot w + b)$
 - $P(\hat{y} = 0|x) = 1 - P(\hat{y} = 1|x) = 1 - \sigma(x \cdot w + b)$
 - $class(x) = \hat{y} = 1(\text{if } P(\hat{y} = 1|x) \geq 0); 0(\text{otherwise})$
- Muti-class Classification:** $class(x) = \hat{y} = \operatorname{argmax}_i \hat{y}_{[i]}$
- Softmax function:** maps output to probability distribution.
$$\operatorname{softmax}(x) = \frac{e^{x_{[i]}}}{\sum_j e^{x_{[j]}}}$$
. Use in multinomial logistic regression.

Training & Loss Functions

- $\hat{\theta} = \operatorname{argmin}_{\theta} (\frac{1}{n} \sum_{i=1}^n L(f(x_i; \theta), y_i) + \lambda R(\theta))$
- Binary Cross-Entropy Loss:**
$$L_{\text{logistic}}(\hat{y}, y) = -y \log_2 \hat{y} - (1 - y) \log_2 (1 - \hat{y})$$
- Categorical Cross-Entropy Loss:**
$$L_{\text{c-e}}(\hat{y}, y) = -\sum_i y_{[i]} \log_2 (\hat{y}_{[i]})$$
- Hard Classification Problem**, one *t* as single correct class:
$$L_{\text{c-e}}(\hat{y}, y) = -\log_2 (\hat{y}_{[t]})$$
- Squared (quadratic) loss:** $L(\hat{y}, y) = \frac{1}{2} (\hat{y} - y)^2$
- Regularization:**
 - $R_{L_2}(W) = ||W||_2^2 = \sum_{i,j} (W_{[i,j]})^2$
 - $R_{L_1}(W) = ||W||_1 = \sum_{i,j} |W_{[i,j]}|$
- Gradient Descent:**
 - $L(\mathbf{w}_1) > L(\mathbf{w}_2) > \cdots > \min$
 - $\delta L(w_0, w_1, \dots, w_{m-1}) = (\frac{\delta L}{\delta w_0}, \frac{\delta L}{\delta w_q}, \dots, \frac{\delta L}{\delta w_{m-1}})$
 - $L(\mathbf{w}, \mathbf{v}) \approx L(\mathbf{w}) + \mathbf{v} \cdot \Delta L(\mathbf{w})$
 - $\mathbf{v} = -\alpha \Delta L(\mathbf{w}); \alpha > 0$
 - $L(\mathbf{w} - \alpha \Delta L(\mathbf{w})) \approx L(\mathbf{w}) - \alpha ||\Delta L(\mathbf{w})||^2$
 - Hence: $L(\mathbf{w}) > L(\mathbf{w} - \alpha \Delta L(\mathbf{w}))$
 - $w_{t+1} = w_t - \alpha_t \Delta L(\mathbf{w}_t)$

Feed-Forward Neural Networks

- XOR Problem.** Can be solved by applying a nonlinear input transformation.
- Multilayer Perceptron (MLP)** with 1 hidden layer:
$$NN_{MLP1}(x) = g(\mathbf{xW}^1 + \mathbf{b}^1)\mathbf{W}^2 + \mathbf{b}^2$$
- Activation Functions:**
 - Sigmoid. Nice property: $\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$
 - Hyperbolic Tangent. $\tanh(x) = \frac{e^{2x}-1}{e^{2x}+1} = \frac{e^x-e^{-x}}{e^x+e^{-x}}$.

Nice property: $\frac{d}{dx} \tanh(x) = 1 - [\tanh(x)]^2$
 - Hard tanh = -1 if *x* < −1; 1 if *x* > 1; x otherwise
 - ReLU = max(0, x)
- Dropout Regularization:** Multiply the output of a hidden layer with random masking vectors (setting random neurons to 0). Prevent overfitting.

Neural Network Training

- Multivariate Chain Rule:**
$$\frac{\delta L}{\delta w_i} = \frac{\delta L}{\delta x_1} \frac{\delta x_1}{\delta w_i} + \frac{\delta L}{\delta x_2} \frac{\delta x_2}{\delta w_i} + \cdots + \frac{\delta L}{\delta x_n} \frac{\delta x_n}{\delta w_i}$$
- Backpropagation. Recursive calculation.
- Computational Graph: A directed acyclic graph. Nodes: mathematical operations or bound variables. Edges: flow of intermediary values between nodes

Word Embeddings (Word2Vec)

- Sparse (One-hot) vs Dense vectors (Learned)
- Two vectors for each word: **center**, **context**
- Make the vector **w** of center word close to vectors **c** of words in its context. Binary prediction task
- Training distinguishes *D* and $\bar{D}$
- Negative words sampled: $\frac{\#(w)^{0.75}}{\sum_{w'} \#(w')^{0.75}}$
 - Treat target word **w** and neighbouring context word **c** as positive examples
 - Randomly sample other words to get negative examples
 - Use **logistic regression** to train a classifier to distinguish +ve and -ve examples
 - $P(D = 1|w, c) = \frac{1}{1+e^{-w \cdot c}}$
 - $P(D = 0|w, c) = 1 - P(D = 1|w, c) = \frac{1}{1+e^{w \cdot c}}$

- Minimise: $L(\theta, D, \bar{D}) = -\sum_{(w,c) \in D} \log P(D = 1|w, c) - \sum_{(w,c) \in \bar{D}} \log P(D = 0|w, c)$
- CBOW: $\log P(D = 1|w, c_{1:k}) = \frac{\log \frac{1}{1+e^{-(w \cdot c_1 + w \cdot c_2 + \dots + w \cdot c_k)}}}{1+e^{-(w \cdot c_1 + w \cdot c_2 + \dots + w \cdot c_k)}}$
- Skipgram: $\log P(D = 1|w, c_w:k) = \sum_{i=1}^k \log \frac{1}{1+e^{-w \cdot c_i}}$
- Gradient Descent. Final embedding: $w_i + c_i$ or w_i
- 4. Used the learned weight vectors as embeddings.
- Cosine similarity: $\text{sim}_{cos}(u, v) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$
- Used for word analogy task via vector algebra

Neural Language Models

- $P(w_i|w_1, w_2, \dots, w_{i-1}) = \frac{P(w_1, w_2, \dots, w_i)}{P(w_1, w_2, \dots, w_{i-1})}$
- $P(w_{1:i}) = P(w_1) \times P(w_2|w_1) \times \dots \times P(w_i|w_{1:i-1})$
- Downsides of n-gram LMs:** requires smoothing; number of n-grams grow multiplicatively; lack of generalization
- Input:** a sequence of k words
- Output:** probability distribution over next word
- $\hat{y} = P(w_i|w_{i:k}) = LM(w_{1:k}) = \text{softmax}((g(\mathbf{x}\mathbf{W}^1 + \mathbf{b}^1))\mathbf{W}^2 + \mathbf{b}^2)$
- Parameters: $\mathbf{E}_{[w]}$, $\mathbf{W}^1$, $\mathbf{b}^1$, $\mathbf{W}^2$, $\mathbf{b}^2$.
- Note: $E \in \mathbb{R}^{|V| \times d_w}$; $x = [v(w_1); v(w_2); \dots; v(w_k)]$
- Training samples: sequence of k words paired with $k + 1^{\text{th}}$ word as target label for classification
- Loss function: Categorical cross-entropy loss

RNN

- Model non-Markovian dependencies. Deep NN.
- $\text{RNN}^*(x_{1:n}; s_0) = y_{1:n}$
- $s_i = R(s_{i-1}, x_i)$; $y_i = O(s_i)$
- $x_i \in \mathbb{R}^{d_{in}}$, $y_i \in \mathbb{R}^{d_{out}}$, $s_i \in \mathbb{R}^{d_{out}}$
- sequence labelling, sequence classification, language modeling, encoder-decoder
- Sentence Level Classification:** $\hat{y} = \text{softmax}(MLP(\text{RNN}(x_{1:n})))$. Cross-entropy loss. Using pre-trained word embeddings.
- Sequence Labelling:** Produce an output $\hat{t}_i$ after each input x_i is read. Loss: $L(\hat{t}_{1:n}, t_{1:n}) = \sum_{i=1}^n L_{\text{local}}(\hat{t}_i, t_i)$
- Language Modelling:** output at a timestep acts as input to the next time step, $y_i = x_{i+1}$. Loss = $L = -\sum_t \log_2(\hat{y}_t)$
- Conditioned Generation: Concatenate embedded input vector with a condition vector, $c \cdot s_j = R(s_{j-1}, [\hat{t}_j; c])$
- Encoder-Decoder:** Sequence to sequence
 - Encoder: $RNN_{enc}^*(x_1, \dots, x_n) = c_1, \dots, c_n$
 - Decoder: Conditioned generator RNN.
- $\bar{a}_{[i]}^j = \begin{cases} s_{j-1} \cdot c_i & \text{dot product} \\ s_{j-1} W c_i & \text{bilinear} \\ \tanh([s_{j-1}; c_i]U + b) \cdot v & \text{MLP} \end{cases}$
- $a^j = \text{softmax}(\bar{a}_{[1]}^j, \dots, \bar{a}_{[n]}^j)$
- $c^j = \sum_{i=1}^n a_{[i]}^j \cdot c_i \leftarrow \text{attend}$
- $s_j = R_{\text{dec}}(s_{j-1}, [\hat{t}_j; c^j])$
- $y_j = O_{\text{dec}}(s_j)$

Transformers

- Exploit parallel processing
- Transformer blocks are stacked on top of one another. Hence, input and output dimensions are the same.

- input** $\rightarrow$ **Self-Att Layer** $\rightarrow$ **Layer Normalise** $\rightarrow$ **Feed-Forward** $\rightarrow$ **Layer Normalise** $\rightarrow$ **output**. Residual connection before each layer normalise.
- Decoder-only, Encoder-only, Encoder-decoder

Decoder-Only Transformer

- Language Modelling, Text Generation (GPT, Llama)
- Positional Embedding** vectors are added to token embeddings.
- Static function to map position integers to real-valued vectors (combination of sine and cosine function)
- Language Modelling Head:** Take last output vector h_N^L and outputs a distribution over vocab. V . $\mathbf{u} = \mathbf{h}_N^L \mathbf{E}^T$. $\mathbf{y} = \text{softmax}(\mathbf{u})$
- Training via teacher forcing, $L = \frac{1}{N} \sum_{t=1}^N L_{CE}$
- Generation sampling:** Greedy, Random, Temperature (divide $\mathbf{u}$ by $\tau \in (0, 1]$), Top-k (renormalise highest k), Top-p (percent)

Single Head Self Attention

- (left-to-right) Self-attention layer. Only past tokens
- Idea: a vector can have 3 different roles: Query, Key, Value
- $\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q$; $\mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K$; $\mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$
- $x_i \in \mathbb{R}^d$; $\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d \times d_k}$; $\mathbf{W}^V \in \mathbb{R}^{d \times d_v}$
- $\text{sim}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}$. Numerical stability
- $\alpha_{ij} = \frac{e^{\text{sim}(x_i, x_j)}}{\sum_{k=1}^i e^{\text{sim}(x_i, x_k)}}$, $\forall j \leq i$ (softmax)
- $\mathbf{y}_i = \sum_{j=1}^i \alpha_{ij} \mathbf{v}_j = \text{SelfAttn}(\mathbf{x}_i)$
- Future masking in QK^T to turn top right triangle half of the matrix to negative infinity.
- Layer Normalisation: normalise values in a vector to have mean = 0 and variance = 1. $\mu = \frac{1}{d} \sum_{x=1}^d x_i$ and $\sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}$
- LayerNorm**($\mathbf{x}$) = $\gamma \frac{\mathbf{x} - \mu}{\sigma} + \beta$. γ and β : learnable params
- Multi-Head Self Attention**
- Different transformer heads to model different "representation subspace" or different relationships
- A head has its own $\mathbf{W}_i^Q, \mathbf{W}_i^K, \mathbf{W}_i^V$ and contributes $\mathbf{Y}^i$
- Final Output: $\mathbf{Y} = (\mathbf{Y}_1 \oplus \mathbf{Y}_2 \oplus \dots \oplus \mathbf{Y}_h) \mathbf{W}^O$
- $\mathbf{Y}_i \in \mathbb{R}^{N \times d_v}$; $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d}$

Encoder-Only Transformer

- BERT(Bidirectional Encoder Representation Transformers)
- No masking of $\mathbf{QK}^T$
- Pretrain:** With unlabeled text for Masked LM and Next Sentence Prediction
- Finetune:** One additional output layer for target NLP task. Finetune pretrained parameters with smaller labelled data
- Pretraining:**
 - Masked LM.** Mask 15% of all input tokens (80% [MASK], 10% random tokens, 10% unchanged). CE Loss.
 - $\mathbf{u}_i = \mathbf{h}_i^L \mathbf{E}^T$; $\mathbf{y}_i = \text{softmax}(\mathbf{u}_i)$. Similarity scoring.
 - $L_{MLM} = -\frac{1}{|M|} \sum_{i \in M} \log P(x_i | \mathbf{h}_i^L)$
- Next Sentence Prediction.** Two sentences A, B. Two-class prediction based on [CLS]: isNext, notNext. CE Loss
- Contextual Embeddings:** h_i^L is actually contextual embedding of x_i in the context of input sentence.

Fine-Tuning:

- Sentence-level task** (Output vector [CLS] token) vs **Token-level task** (Output vector for ith token)
- Sentence-level: Sentiment Classification. Natural Language Inference (premise, hypothesis: entail/contradict/neutral)
- Token-level: POS tagging, Named entity recognition (NER) [PER, ORG, LOC, GPE]
- NER tagging schemes:** IO, BIO, BIOES. (e.g B-PER). NER Head takes output vector, softmax, argmax.
- Extractive Question Answering.** Input: passage, question. Output: Extracted segment of text (span) in the passage as answer.
- Utilises **span labelling**. Add a linear layer to BERT with two vectors $\mathbf{s}$ and $\mathbf{e}$.
- $P_s(i) = \frac{e^{\mathbf{s} \cdot \mathbf{p}_i}}{\sum_{j=1}^m e^{\mathbf{s} \cdot \mathbf{p}_j}}$; $P_e(i) = \frac{e^{\mathbf{e} \cdot \mathbf{p}_i}}{\sum_{j=1}^m e^{\mathbf{e} \cdot \mathbf{p}_j}}$
- Loss Function: $L = -\log P_s(a_s^*) - \log P_e(a_e^*)$
- Score of candidate span: $\mathbf{s} \cdot \mathbf{p}_i + \mathbf{e} \cdot \mathbf{p}_j$. Highest scoring span where $i \leq j$. [CLS] as no ans. Overlapping windows.

Post-Training (Supervised Fine-Tuning)

- Model Alignment: Instruction fine-tuning. Preference-based learning.
- Bradley-Terry Model: Preference Reward Function**
 - $\delta = z_i - z_j = r(x, o_i) - r(x, o_j)$
 - $L_{CE} = \mathbb{E}_{(x, o_w, o_l)} D[-\log \sigma(r(x, o_w) - r(x, o_l))]$
 - $P(o_i \succ o_j | x) = \sigma(z_i - z_j) = \sigma(\delta) = \frac{1}{1+e^{-\delta}}$
 - $\delta = \log \frac{P(o_i \succ o_j | x)}{P(o_j \succ o_i | x)}$
 - Replace LM head with linear layer to output scalar score
- Reward function used in RLHF to learn an aligned LLM
- Direct Preference Optimization:** Train a policy (LLM) π_θ : maximises rewards for responses from the policy, but penalises models that diverge from SFT model π_{ref}
- $\pi^* = \text{argmax}\{\mathbb{E}_{D, o! \pi_\theta(o|x)} [r(x, o)] - \beta \mathbb{D}_{KL}[\pi_\theta(o|x) || \pi_{ref}(o|x)]\}$. $\beta > 0$: penalty Temperature
- KL Divergence: $\mathbb{D}_{KL}[P||Q] = \sum_x P(x) \log \frac{P(x)}{Q(x)}$
- Yields: $r(x, o) = \beta \log \frac{\pi_\theta(o|x)}{\pi_{ref}(o|x)} + \beta \log Z(x)$
- $L_{DPO}(\pi_\theta) = \mathbb{E}_{(x, o_w, o_l)} D[-\log \sigma(\beta \log \frac{\pi_\theta(o_w|x)}{\pi_{ref}(o_w|x)} - \beta \log \frac{\pi_\theta(o_l|x)}{\pi_{ref}(o_l|x)})]$
- Simpler (no need for reward model), stable, efficient

Encoder-Decoder Transformers (MT, GEC)

Cross-Attention

- Decoder has extra cross-attention layer with weights: $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$
- Q : based on output of previous layer of decoder, $\mathbf{X}_{\text{dec}}$
- K, V based on final output of the **top** layer of encoder, $\mathbf{X}_{\text{enc}}$
- $Q = X_{\text{dec}} W^Q$, $K = X_{\text{enc}} W^K$, $V = X_{\text{enc}} W^V$
- Training: Teacher forcing. $L = \frac{1}{N} \sum_{i=1}^N (-\log P(y_{i+1}))$
- Encoder output does not contribute to loss function
- Decoding: Search for most probable (translation) output $T^* = \text{argmax}_{t_1, \dots, t_N} \prod_{i=1}^N \log P(t_i | t_1, \dots, t_{i-1}, s)$ Stop generating when $t_N = EOS$ or $N > N_{max}$
- Greedy, Beam-Search (keep most probable k tokens at each time step, k complete paths)

- Minimum Bayes Risk (Translation most similar to candidate translations) $T^* = \text{argmax}_{T \in \tau} \sum_{C \in \tau} \text{sim}(T, C)$

chrF

- Measure number of character n-gram overlap
- HYP:** Machine Translation (P). **REF:** Human Translation (R)
- $F_\beta = \frac{(1+\beta^2 \times P \times R)}{(\beta^2 \times P) + R}$
- R is weighted β times as much as P
- Find unigrams, ... matches between HYP and REF
- Unigram P, Bigram P, Unigram R, Bigram R
- chrP** and **chrR**: % k-grams

BLEU

- Match candidate translation with multiple references in combination to get a modified unigram (word) precision.
- $p_n = \frac{\sum_{C \in \text{Candidates}} \sum_{\text{ngram} \in C} \text{count}_{\text{clip}}(\text{ngram})}{\sum_{C' \in \text{Candidates}} \sum_{\text{ngram} \in C'} \text{count}_{\text{clip}}(\text{ngram}^T)}$
- $BP = 1$ if $c > r$; $BP = e^{(1-r/c)}$ if $c \leq r$
- $BLEU = BP \times e^{\frac{1}{N} \sum_{n=1}^N \log p_n}$

BERTScore

- Embed tokens contextually using BERT
- Compute cosine similarity between every token in C and every token in R
- Perform greedy matching and calculate F-score.
- $Pr = \frac{1}{|C|} \sum_{c \in C} \max_{r \in R} (\mathbf{v}_r \cdot \mathbf{v}_c)$
- $Rc = \frac{1}{|R|} \sum_{r \in R} \max_{c \in C} (\mathbf{v}_r \cdot \mathbf{v}_c)$

IR and RAG

- $p(x_{1:n}) = \prod_{i=1}^n p(x_i | R(q); \text{prompt}; [Q :]; q; [A :]; x_i)$
- tf-idf weighting**
- $\text{count}(t, d)$: number of times term t appear in document d
- $t f_{t,d} = 1 + \log_{10} \text{count}(t, d)$ if $\text{count}(t, d) > 0$
- $t f_{t,d} = 0$ otherwise.
- $d f_t$: number of documents that term t appears in
- N : total number of document
- $i d f_t = \log_{10} \frac{N}{d f_t}$
- $t f \cdot i d f(t, d) = t f_{t,d} \times i d f_t$
- d : Sparse vector. $[t f \cdot i d f(t_1, d) \dots t f \cdot i d f(t_{|V|}, d)]$
- $\text{score}(q, d) = \cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|}$
- Inverted index: $\text{termidf} \rightarrow \text{docID}[\text{count}] \rightarrow \text{docID}[\text{count}] \rightarrow$

IR Evaluation

- T: set of retrieved documents (in rank order)
- U: set of relevant documents
- R: subset of retrieved documents that are relevant
- Precision = $\frac{|R|}{|T|}$. Recall = $\frac{|R|}{|U|}$
- Interpolated precision: $p_{int}(r) = \max_{r' \geq r} p(r')$ for $r \in \{0, 0.1, \dots, 1.0\}$
- Average precision AP = $\frac{1}{|R|} \sum_{d \in R} p_r(d)$ (relevant d only)
- $p_r(d)$: precision at the rank d was found
- IR with Dense Vectors**
- One encoder and two encoder approach
- $z = \text{BERT}([CLS]; q; [SEP]; d)[CLS]$
- $z_q = \text{BERT}_Q([CLS]; q)[CLS]$
- $z_d = \text{BERT}_D([CLS]; d)[CLS]$
- $\text{score}(q, d) = \sigma(\mathbf{u} \cdot \mathbf{z})$ or $\text{score}(q, d) = \mathbf{z}_q \cdot \mathbf{z}_d$

POS Tagging

- $\hat{\tau} = \text{argmax}_{t_1:T} \{\prod_{i=1}^T P(t_i | t_{i-1}) \cdot P(w_i | t_i) \cdot P(< /s > | t_T)\}$
- $P(t_i | t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})} P(w_i | t_{i-1}) = \frac{C(t_i, w_i)}{C(t_i)}$

- Direct Evaluation: $O(T \cdot N^T)$
- Viterbi Algorithm: $O(T \cdot N^2)$
- $v_t(j) = \max_{1 \leq i \leq N} v_{t-1}(i) \cdot a_{ij} \cdot b_j(o_t)$
- $a_{ij} \rightarrow P(t_j|t_i); b_j(o_t) \rightarrow P(w_t|t_t)$
- $P(w_i|t_i) = P(\text{unknownword}|t_i) \cdot P(\text{capital}|t_i) \cdot P(\text{ending/hyph}|t_i)$

Context Free Grammar

- Notation: $[_{\text{verb}} \text{ prefer}]$
- Strong equivalence: $L(G1) = L(G2)$ and has same phrase structure
- Weak equivalence: Generate same set of strings but do not assign same phrase structure to each string
- **Chomsky Normal Form (CNF):**
 - The grammar is ϵ -Free
 - Each production of the grammar is either: $A \rightarrow BC$ or $A \rightarrow \alpha$ (i.e either 2 non-terminal symbols or 1 terminal symbol on RHS)
- CFG can be converted into weakly equivalent CFG in CNF.
- Unit production: $A \rightarrow B$ where RHS consists of a single non-terminal symbol B
 1. Construct set P' . Include all non-unit productions.
 2. For non-terminal symbols A and B , if $A \rightarrow A1 \rightarrow \dots \rightarrow B$, then for each non-unit production $B \rightarrow \alpha$. Add $A \rightarrow \alpha$ to P'
 3. Consider each production P' of the form $A \rightarrow X_1X_2 \dots X_m$ where $m \geq 2$. If X_i is a terminal symbol a , introduce a new non-terminal symbol C and a production $C \rightarrow a$. Replace X_i with C.
 4. For each production of the form $A \rightarrow B1B2B3$ replace production by $A \rightarrow B1D1$ and $D1 \rightarrow B2B3$. Right heavy tree.
- **Cocke-Kasami-Younger (CKY) Algorithm:**
 1. **Table:** cols are words, top right half, idx between words.
 2. **Order:** Left column to right, bottom to up.
 3. **Cell ranges:** [0, 1], [0, 2], [0, 3]; [1,2] [1,3]; [2,3]
 4. From the CFG rules, fill bottom cell with non-terminal symbols that has $\text{word}_{\text{target}}$ on the RHS (base case)
 5. Go through each [i,k] + [k,j] combination, see if any non-terminal symbols imply combination of L and R symbols (RHS) write it down in the cell (**OR**)
 6. Consider only trees with **S** at the root
- Scoring Parse Trees: BERT, Span difference, FFN
- $s_{\text{best}}(i, j) = \max_l [(s, j, l)]$ if $j - 1 = 1$
- $s_{\text{best}}(i, j) = \max_l [(s, j, l)] + \max_{i < k < j} [s_{\text{best}}(i, k) + s_{\text{best}}(k, j)]$ if $j - 1 > 1$
- labelled recall R = $\frac{\text{num correct constituents of parser parse}}{\text{constituents in treebank parse}}$
- labelled precision P = $\frac{\text{num correct constituents of parser parse}}{\text{constituents in parser parse}}$
- labeled $F_1 = \frac{2RP}{R+P}$
- Correct: Constituent in treebank parse spans same words with same non-terminal symbol